CLOUDHOST

AWS Web Hosting and Secure Storage Portal

Cloud Computing Lab Project Report

Project Member: Aniket P. Nawkar A7 65 , Pravesh Shrivastava A7 59

Project URL: http://ec2-184-195-187-216.compute-1.amazonaws.com

EC2 Public IP: http://184.195.187.216/

Student / Team	Aniket P. Nawkar , Pravesh Shrivastava
Programme	B.Tech Computer Science & Engineering
Course	Cloud Computing Laboratory

1. Project Overview

CloudHost is a cloud-hosted web portal that demonstrates web application hosting and file management using AWS services. Users can open the portal, view a system dashboard, upload files, and review the records presented by the application.

The supplied screenshots show the application interface and selected AWS Console views for compute, storage, and networking. This report uses the provided project report as the technical reference and distinguishes visible evidence from architecture components described in that reference.

Problem Statement and Objectives

A web application needs a reliable hosting environment and a separate storage layer for uploaded files. CloudHost demonstrates a cloud-based alternative to hosting everything on a local machine.

- Host the web portal on Amazon EC2.
- Provide dashboard and file-upload interfaces.
- Use Amazon S3 for object storage and a VPC for networking.
- Document the implementation flow and visible deployment evidence.

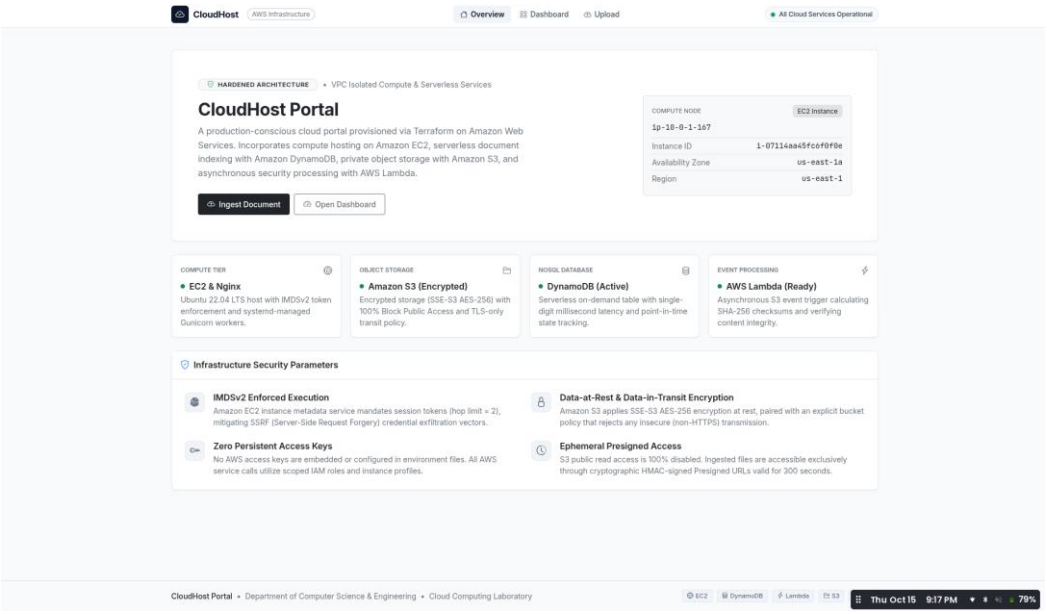


Figure 1.1. CloudHost landing page and portal overview.

2. Architecture and Deployment Environment

The reference design describes a multi-tier architecture: EC2 hosts the web application, S3 stores uploaded objects, DynamoDB stores file metadata, and Lambda processes S3 upload events. A VPC, subnet, Internet Gateway, security groups, IAM permissions, and Terraform support the infrastructure and access-control layers.

Amazon EC2	Runs the web application	Browser → web server
Amazon S3	Stores uploaded objects	Upload → object storage
Amazon DynamoDB	Stores file metadata in the reference design	Metadata → dashboard
AWS Lambda	Processes upload events in the reference design	S3 event → hash/update
VPC / Internet Gateway	Network isolation and routing	Subnet → route → internet
Terraform	Defines repeatable infrastructure	Configuration → resources

Deployment evidence

The screenshots below show the EC2 instance details and its monitoring view. These views document the compute environment; they do not alone prove every backend integration is active.

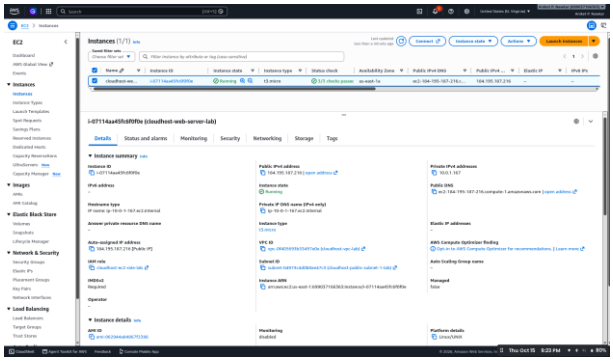


Figure 2.1. EC2 instance details.

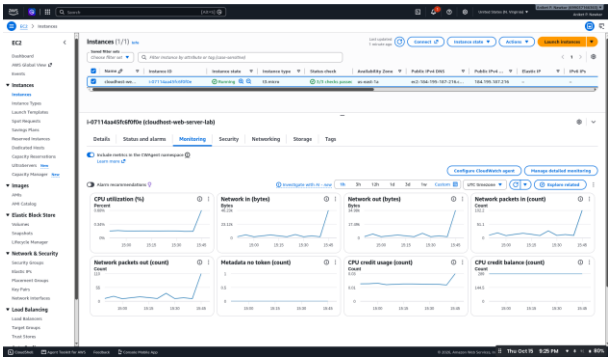


Figure 2.2. EC2 monitoring metrics.

Typical request path: a browser reaches the EC2-hosted web server through the configured network rules. The application handles the request and uses its AWS permissions to interact with storage or metadata services as configured.

3. Application Interface and User Workflow

3.1 Landing page and dashboard

The landing page introduces CloudHost and provides navigation to the system dashboard and file upload. The dashboard displays file-related summary metrics and a table of records, including status information visible in the supplied output.

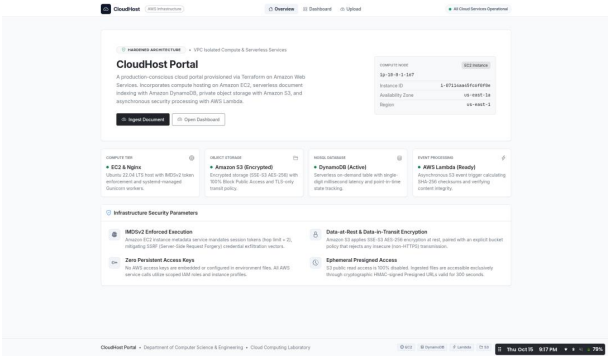


Figure 3.1. Portal landing page.

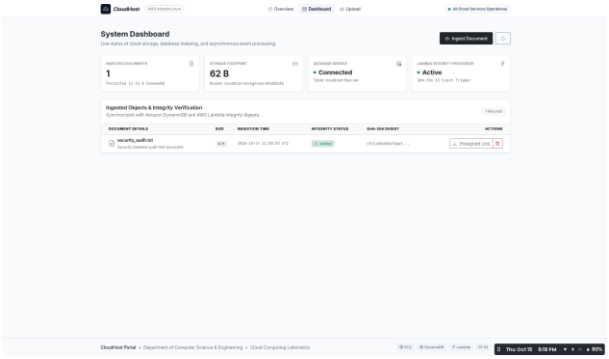


Figure 3.2. System dashboard.

3.2 File upload

The upload interface provides a file-selection/drop area and a recent-record panel. The intended user sequence is to select a sample file, submit it through the form, and inspect the resulting record in the dashboard. Use non-sensitive test files during demonstration.

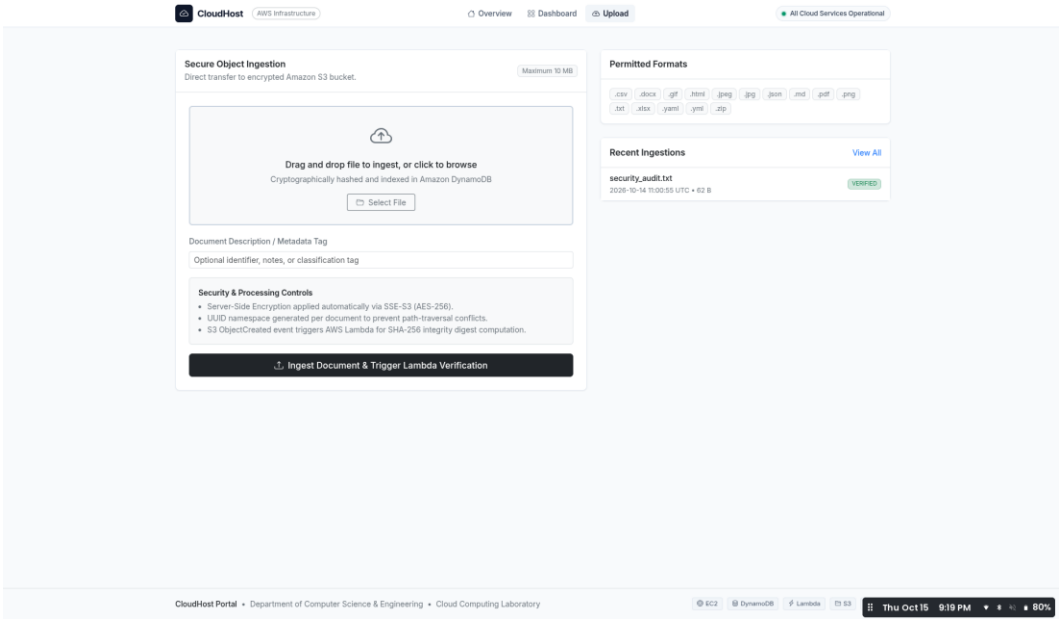


Figure 3.3. File upload interface and recent-record panel.

User-level workflow

- Open the deployed portal in a browser.
- Navigate to the dashboard to review the displayed state.
- Open the upload page and select a sample file.
- Submit the upload and return to the dashboard to check the record.

4. AWS Infrastructure and Storage Configuration

The supplied AWS Console screenshots document selected infrastructure views: EC2 configuration and networking, a VPC view, and an S3 bucket listing. They provide evidence of the cloud environment associated with the application.

4.1 Network and compute layer

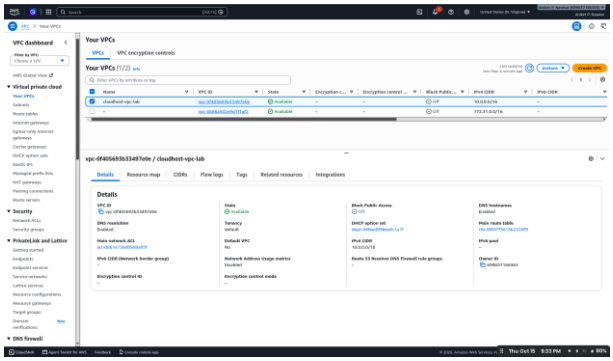


Figure 4.1. VPC configuration view.

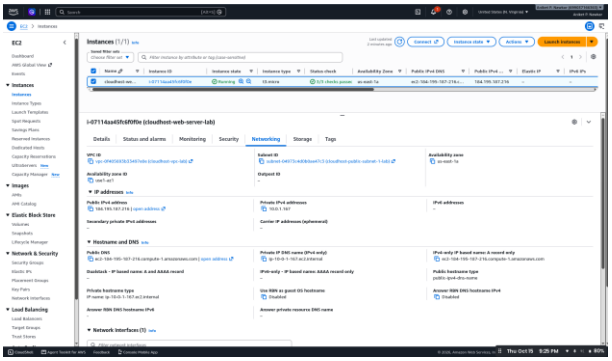


Figure 4.2. EC2 networking details.

The VPC provides a logically isolated network. Subnets, route tables, an Internet Gateway, and security groups determine how the instance is reached. Inbound access should be limited to the ports and source addresses needed for the application and administration.

4.2 Object storage layer

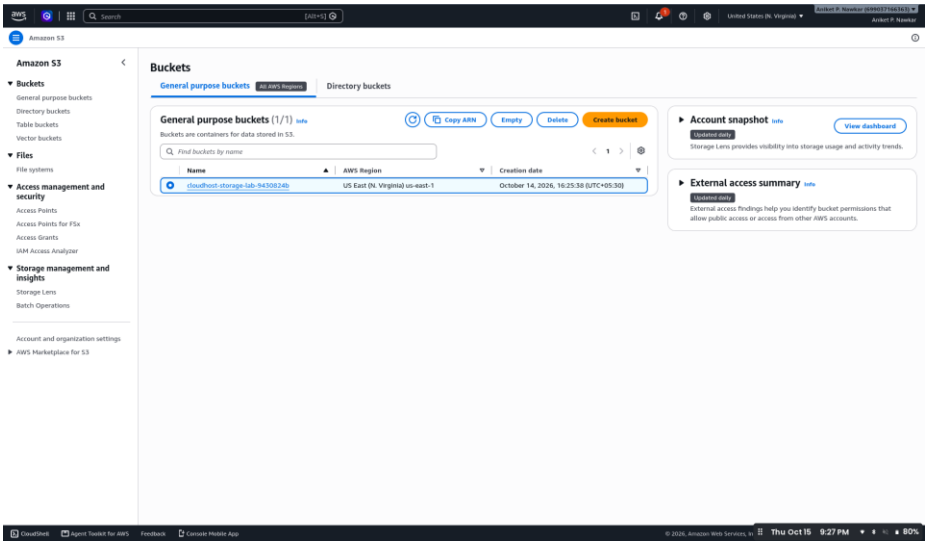


Figure 4.3. Amazon S3 bucket listing in the

*AWS Console.*In the reference architecture, S3 stores uploaded file binaries separately from application metadata. Keep Block Public Access enabled unless public access is explicitly required, and verify bucket policies before using real files.

4.3 Security considerations

- Prefer IAM roles and temporary credentials over hard-coded access keys.

- Use least-privilege policies for EC2, S3, DynamoDB, and Lambda.
- Apply encryption and secure transport controls where configured.
- Review running resources after testing to prevent unexpected charges.

5. End-to-End Implementation Flow and Results

5.1 Implementation sequence

1. Provision — Infrastructure configuration defines compute, network, storage, and access-control resources.

2. Host — The application runs on EC2 and is served to a browser.

3. Interact — The user opens the portal, views the dashboard, and submits a file.

4. Store — The reference design stores the binary in S3 and metadata in DynamoDB.

5. Process — In the reference design, an S3 object-created event invokes Lambda to calculate a SHA-256 digest and update the metadata record.

6. Verify — The user checks the dashboard; service health and backend results should be confirmed with direct test evidence.

5.2 Verification summary

Homepage	Application screenshot	Homepage rendered
Dashboard	Dashboard screenshot	Dashboard rendered with summary and record table
Upload UI	Upload screenshot	Upload form rendered
EC2	Instance / monitoring views	EC2 configuration views are available
S3	Bucket listing	S3 bucket listing is visible
VPC	VPC console view	VPC details are visible

Conclusion

CloudHost demonstrates a browser-accessible cloud portal alongside AWS compute, storage, and networking resources. The screenshots establish the visible application interface and selected AWS Console configuration. To claim complete end-to-end file verification, include direct evidence of the uploaded S3 object, its DynamoDB metadata record, and the corresponding Lambda execution/hash result if those components are enabled.

Reference

Technical reference: CloudHost_AWS_Project_Report.docx supplied with the request. Implementation evidence: screenshots in the supplied project-output ZIP archive.